MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of CSE(Artificial Intelligence and Machine Learning)

BACKEND WEB TECHNOLOGIES (BWT) HACKATHON

PROJECT DOCUMENTATION

Project: CertiVerify — Digital Certificate Verification Platform

Field	Details
Project Title	CertiVerify — Digital Certificate Verification Platform (Problem Statement #79: Digital Certificate Verification)
Team Name / Team No.	Problem Statement #79 / Team No.: 05_____
Year / Semester / Section	II Year B.Tech / 3rd Sem / F Section
Technology Stack	MongoDB, Express.js, React.js, Node.js (MERN), REST APIs
Faculty Mentor	Alekhya Ma'am & Shivmani Sir

Team Members

S. No.	Name of Student	Roll No.	Role
1	Manthan Biradar	25MVCSMR0512	Team Lead
2	Yashvardhan Singh Gaur	25MVCSMR0540	Backend Development
3	Jampala Purnasena	25MVCSMR0491	Backend Development
4	Biswajit Behera	25MVCSMR0472	Frontend Development

1. Problem Statement

Organizations issue educational, training, and achievement certificates that may need to be checked by employers, institutions, or other authorized verifiers. Manual verification can be slow and may require contacting the issuing organization. CertiVerify is a MERN-stack web application designed to let authorized organizations issue and manage certificate records, while allowing verifiers to check a certificate using its unique certificate ID or QR code.

Input: Student/recipient details, course or achievement name, certificate ID, issue date, and certificate status entered by an authorized organization.

Output: A certificate record and a verification result showing the status recorded by the system, such as valid, revoked, expired where expiry is implemented, or not found.

Constraints: Certificate management operations must be authenticated and authorized on the server. Public verification should disclose only the information needed to confirm the record. The result confirms a match with the system's stored record; it does not independently guarantee the legitimacy of an issuer outside the platform.

2. Objectives

- Build a full-stack digital certificate issuance and verification application using the MERN stack.
- Store and manage certificate and related organization/recipient records in MongoDB.
- Develop REST APIs using Node.js and Express.js for the main certificate operations.
- Create a React-based user interface for authorized organizations and public verifiers.
- Implement authentication and server-side role-based authorization where required.
- Support certificate lookup using a unique certificate ID and QR-code-based verification.
- Demonstrate certificate lifecycle management, including status updates and revocation where implemented.
- Apply input validation, safe configuration practices, and access-control checks.

3. Proposed Approach

CertiVerify follows a client-server architecture. The React frontend presents dashboards, forms, and the public verification interface. The Express/Node.js backend validates requests, applies authentication and authorization rules, performs certificate business logic, and communicates with MongoDB. Each issued certificate is associated with a unique identifier that can be entered manually or encoded in a QR code.

The main workflow is: an authorized organization signs in, creates a certificate record, and receives a certificate ID/QR code; a verifier scans the QR code or enters the ID; the backend looks up the current record and returns a limited verification response. If a certificate is revoked, subsequent verification should reflect the updated status.

Authentication and authorization are enforced on the backend rather than relying only on frontend navigation. Database queries should be scoped to the authenticated organization where records are private. Secrets are configured through environment variables and must not be committed to a public repository.

4. Algorithm / Pseudocode

1. Receive a certificate creation request from an authenticated organization.
2. Validate required fields such as recipient/student, course, issue date, and permitted status values.

3. Verify the user's role and organization permissions on the server.

4. Generate or assign a unique certificate identifier and create the database record.

5. Generate a QR code that points to the certificate's verification route, if enabled.

6. For a verification request, accept a certificate ID or QR-linked identifier.

7. Look up the matching certificate record in MongoDB using a safe query.

8. If no record exists, return a not-found result; otherwise return the current allowed status and limited public details.

9. For a revocation request, require authorization, update the record, and preserve an audit trail if supported.

10. Return a clear response to the React interface and display the result to the user.

5. Complexity Analysis

Let N be the number of certificate records and K be the number of fields returned for a record. Actual performance depends on schema design, indexes, database deployment, and query shape. With an index on the unique certificate ID, a typical lookup is approximately O(log N) in a B-tree index, while an unindexed scan can be O(N). Creating or updating one certificate is typically dominated by the database write and index maintenance. QR-code generation depends on the encoded data length and library implementation.

Operation	Typical complexity (indexed)	Notes
Certificate lookup by unique ID	O(log N) approximate	Requires an appropriate database index.
Unindexed certificate lookup	O(N)	May scan many stored records.
Create/update certificate	Database-dependent	Includes validation, write, and index maintenance.
List/search certificates	Depends on query and result size	Pagination and indexes help control cost.

6. Implementation

The application is organized into a React frontend and a Node.js/Express backend. The frontend contains pages and reusable components for authentication, dashboards, certificate forms, status display, and public verification. The backend contains API routes, controllers, middleware, data models, and services for certificate operations and QR/PDF-related functionality where enabled.

Main technologies and responsibilities:

- React.js: renders the user interface and sends requests to the backend API.
- Node.js and Express.js: implement REST endpoints, request handling, business rules, and middleware.
- MongoDB with Mongoose: stores certificate and related application records.
- Authentication middleware: identifies signed-in users and protects private routes.
- Role/ownership checks: limit sensitive operations and private records to permitted users.
- QR-code and PDF services: support certificate verification links and certificate output where implemented.
- Environment configuration: keeps database connection strings and authentication secrets outside committed source code.

The final repository should include source code, package manifests and lockfiles, setup documentation, and example environment files with placeholders. Real .env files, credentials, node_modules, and generated build output should be excluded from the public repository.

7. Results and Test Cases

The following test cases are recommended for validating the submitted build. Record actual outcomes from the final application run; do not report a test as passed unless it has been executed.

Test case	Expected result	Actual result
Authorized organization login	Valid credentials permit access; invalid credentials are rejected.	To be recorded
Issue a certificate	Valid input creates a record with a unique certificate ID.	To be recorded
Verify existing certificate	Correct ID/QR returns the current record status.	To be recorded
Unknown certificate ID	System returns a clear not-found result.	To be recorded
Revoke certificate	Authorized revocation updates the stored status and later verification reflects it.	To be recorded
Unauthenticated protected request	Backend rejects access to protected operations.	To be recorded
Cross-organization access	One organization cannot read or modify another organization's private records.	To be recorded
Input validation	Malformed or disallowed input is rejected safely.	To be recorded

Insert screenshots of the working application and replace 'To be recorded' with actual results before final submission. Do not claim that security, build, or integration tests passed unless the corresponding checks were run.

8. Conclusion and Future Scope

CertiVerify applies the MERN stack to the problem of issuing, managing, and verifying digital certificates. Its central workflow connects authorized certificate creation with ID/QR-based public lookup of the current stored status. The project demonstrates frontend development with React, backend REST API development with Node.js and Express.js, and data management using MongoDB.

Future enhancements may include:

- Deployment with production-grade monitoring, backups, and recovery procedures.

- Email notifications for certificate issuance and revocation.
- Bulk certificate issuance and CSV import with strict validation.
- More comprehensive audit logs and organization onboarding/verification.
- Automated end-to-end tests and security testing in an isolated environment.
- Optional digital signatures or integrations with trusted issuers to strengthen trust beyond a database lookup.

9. References

[1] MongoDB, “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/

[2] Express.js, “Express documentation.” [Online]. Available: https://expressjs.com/

[3] React, “React documentation.” [Online]. Available: https://react.dev/

[4] Node.js, “Node.js documentation.” [Online]. Available: https://nodejs.org/docs/

[5] OWASP Foundation, “OWASP Cheat Sheet Series.” [Online]. Available: https://cheatsheetseries.owasp.org/

[6] Mongoose, “Mongoose documentation.” [Online]. Available: https://mongoosejs.com/docs/